

202. Happy Number

Solved ✓

Easy

Topics

Companies

Write an algorithm to determine if a number n is happy.

A **happy number** is a number defined by the following process:

- Starting with any positive integer, replace the number by the sum of the squares of its digits.
- Repeat the process until the number equals 1 (where it will stay), or it **loops endlessly in a cycle** which does not include 1.
- Those numbers for which this process **ends in 1** are happy.

Return `true` if n is a happy number, and `false` if not.

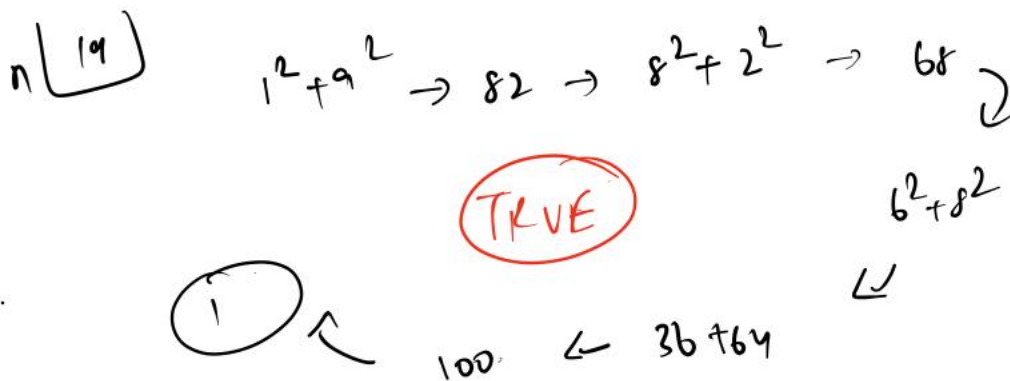
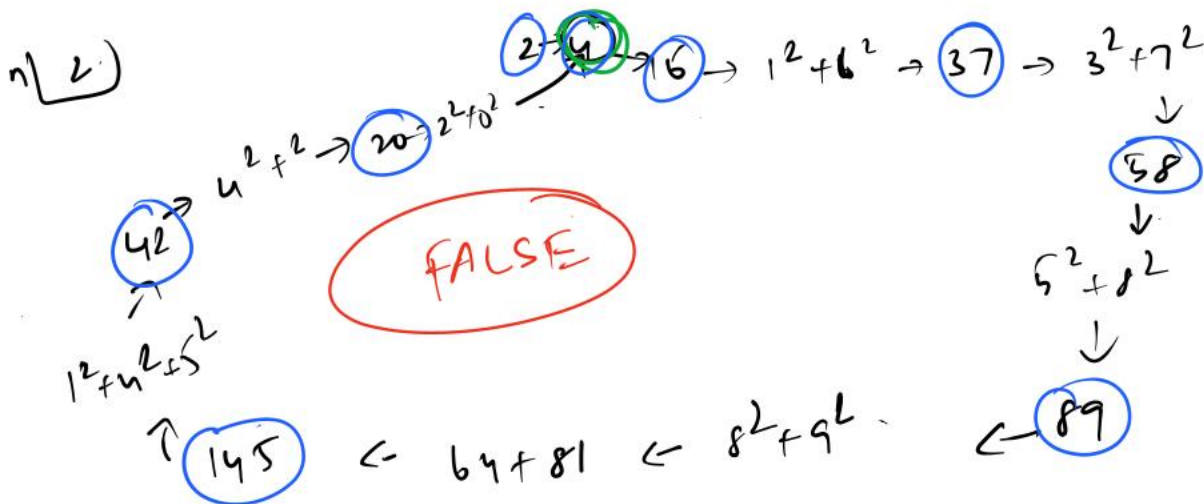
$$15 \rightarrow 1^2 + 5^2$$

$$\downarrow$$

$$26$$

$$\downarrow$$

$$2^2 + 6^2$$



$$1 \rightarrow 1$$

$$2 \rightarrow 2^2 + 7^2$$

$n \mid 251$

$2 + 5 + 1 = 8$

digit $\begin{matrix} 2 \\ 5 \\ 7 \end{matrix}$ total $\begin{matrix} 68 \\ 64 \\ 49 \\ 0 \end{matrix}$

$n \mid \begin{matrix} 2 \\ 25 \\ 257 \end{matrix}$

```
class Solution:
    def isHappy(self, n: int) -> bool:
        seen = set()
        while n != 1:
            if n in seen:
                return False
            seen.add(n)
            total = 0
            while n > 0:
                digit = n % 10
                total += digit * digit
                n //= 10
            n = total
        return True
```

T.C $\rightarrow O(\log n)$ or $O(k)$
S.C $\rightarrow O(\log n)$

$(k) \rightarrow$ no. of digits

0	1	2	3	4	5
32	43	19	72	29	35

↑ ↑ ↑ ↑ ↑ ↑

den $\mid 72$

(11)

$(4) \rightarrow (3)$



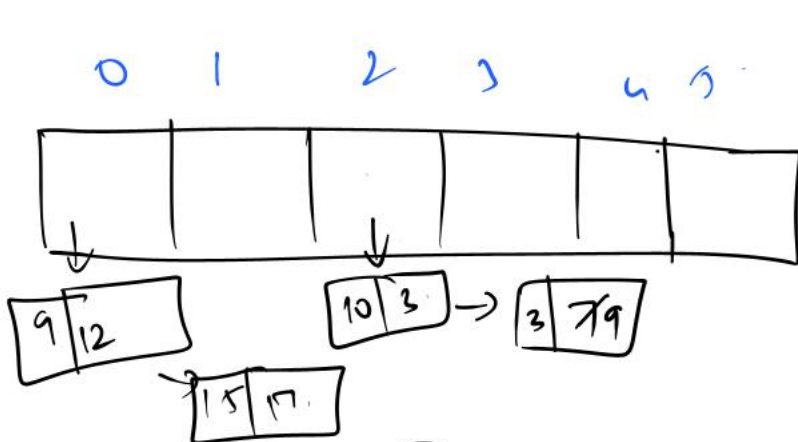
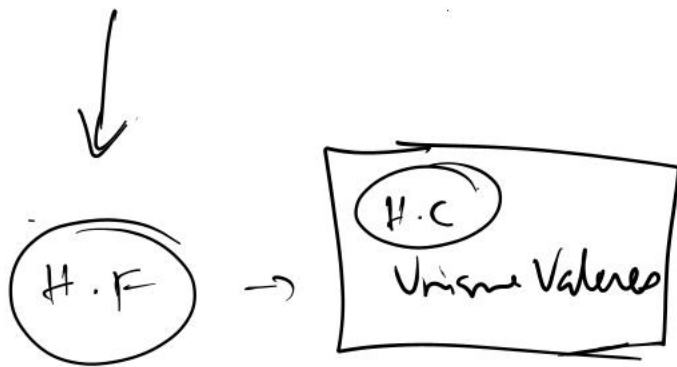
Set()

Hash Map or Dictionary()

$O(1) \rightarrow$ Add, Get, Delete

Array or Linked List

Integer $\rightarrow$ Index



10 : 3
3 : 7
9 : 12
15 : 17
3 : 9

10 $\rightarrow$ HF $\rightarrow$ 2

3 $\rightarrow$ HF $\rightarrow$ 2

9 $\rightarrow$ HF $\rightarrow$ 0

15 $\rightarrow$ HF $\rightarrow$ 0

3 $\rightarrow$ HF $\rightarrow$ 2

work:
key
value
next
index

242. Valid Anagram

Solved 

Easy

Topics

Companies

Given two strings s and t, return true if t is an anagram of s, and false otherwise.

Example 1:

Input: s = "anagram", t = "nagaram"

Output: true

anagram
nagaram

not

not → false

not

True | (aaa)
(aaa)

anagram

abac is

a a @ b - t

a : x x x ⊖
b : 1
c : 1

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t):
            return False
        freq = {}
        for ch in s:
            if ch in freq:
                freq[ch] += 1
            else:
                freq[ch] = 1
        for ch in t:
            if ch not in freq:
                return False
            freq[ch] -= 1
            if freq[ch] < 0:
                return False
        return True
```

→ T.C → $O(n)$

→ S.C → $O(1)$

383. Ransom Note

Solved ✓

Easy

Topics

Companies

Given two strings `ransomNote` and `magazine`, return `true` if `ransomNote` can be constructed by using the letters from `magazine` and `false` otherwise.

Each letter in `magazine` can only be used once in `ransomNote`.

Example 1:

Input: ransomNote = "a", magazine = "b"
Output: false

"aac."
↑
ransomNote

"abbc"
↑
magazine

→ True

a: 1
b: 2
c: 1

```
class Solution:
```

```
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
```

```
        freq = {}
```

```
        for ch in magazine:
```

```
            if ch in freq:
```

```
                freq[ch] += 1
```

```
            else:
```

```
                freq[ch] = 1
```

```
        for ch in ransomNote:
```

```
            if ch not in freq or freq[ch] == 0:
```

```
                return False
```

```
            freq[ch] -= 1
```

```
        return True
```

↓
T.C → $O(n)$

↓
S.C → $O(1)$

387. First Unique Character in a String

Solved ✓

Easy

Topics

Companies

Given a string s, find the first non-repeating character in it and return its index. If it **does not** exist, return -1.

Example 1:

Input: s = "leetcode"

Output: 0

0 1 2 3 4
a a b b c d e
4 0 1 8

```
class Solution:
    def firstUniqChar(self, s: str) -> int:
        freq = {}
        for ch in s :
            if ch not in freq :
                freq[ch] = 1
            else :
                freq[ch] += 1
        for i in range(len(s)) :
            if freq[s[i]] == 1 :
                return i
        return -1
```

T.C $\rightarrow O(n)$

S.C $\rightarrow O(1)$

128. Longest Consecutive Sequence

Medium

Topics

Companies

100 4 200 1 3 2

Given an unsorted array of integers `nums`, return the length of the longest consecutive elements sequence.

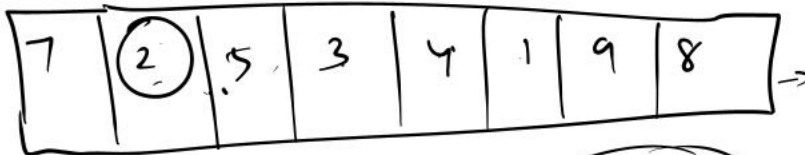
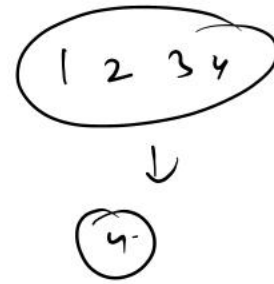
You must write an algorithm that runs in $O(n)$ time.

Example 1:

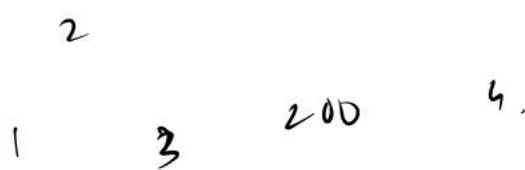
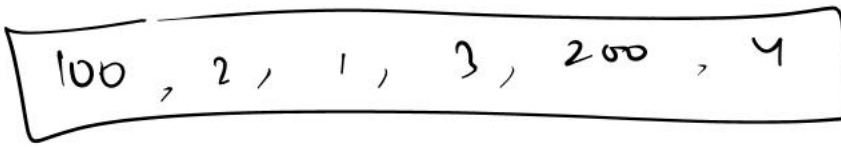
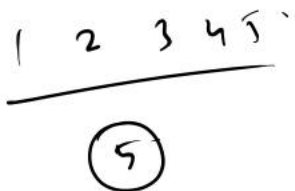
Input: `nums = [100, 4, 200, 1, 3, 2]`

Output: 4

Explanation: The longest consecutive elements sequence is `[1, 2, 3, 4]`. Therefore its length is 4.



o/p: 5



class Solution:

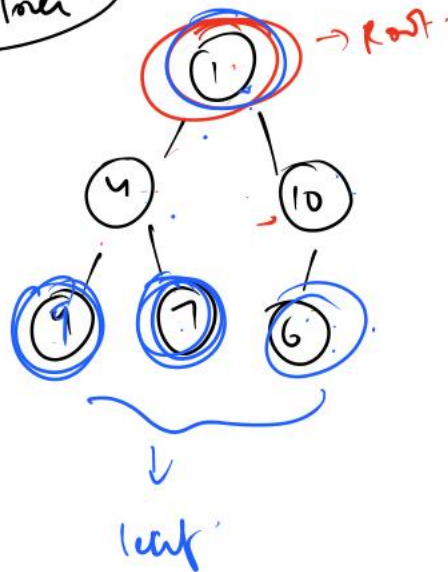
class Solution:

```
def longestConsecutive(self, nums: List[int]) -> int:
    output = 0
    seen = set(nums)
    for elem in seen :
        if elem - 1 not in seen :
            length = 1
            current = elem
            while current + 1 in seen :
                current += 1
                length += 1
            output = max(output, length)
    return output
```

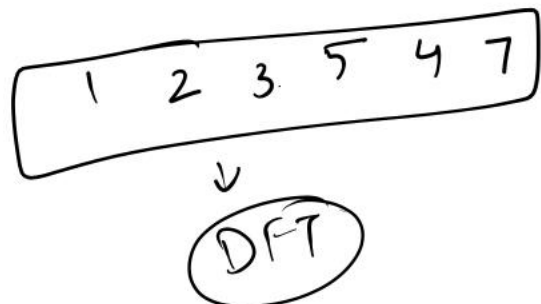
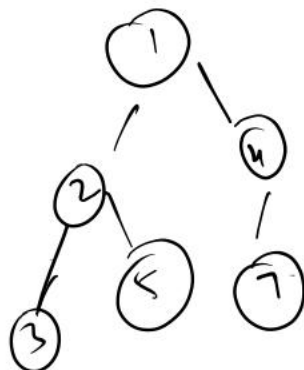
→ T.C. → $O(n)$

→ S.C. → $O(n)$

Tree → Binary Tree



Depth Wise Traversal
Breadth Wise Traversal.



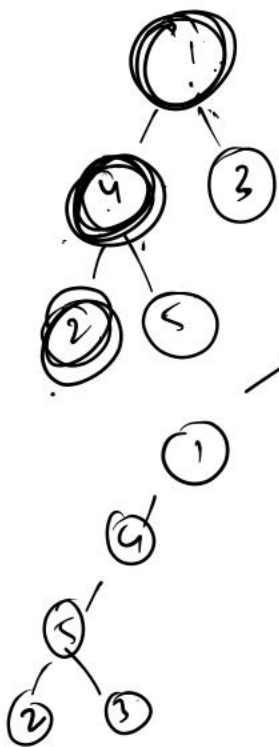
1 2 4 3 5 7. → BFT

Depth First
Traverse

InOrder → left, root, right → 2 4 5 1 3

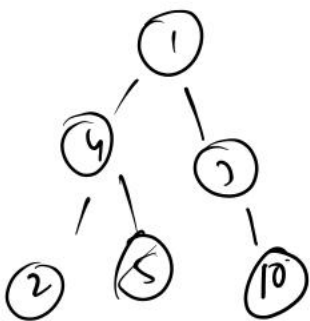
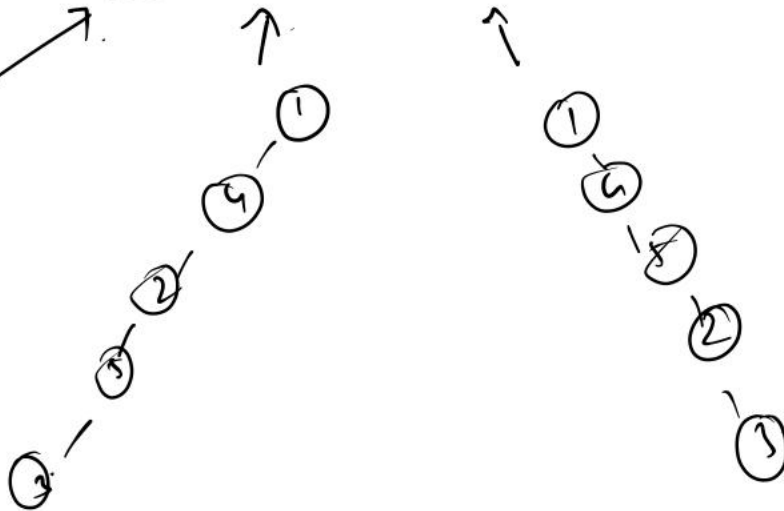
Pre order → Root, left, right → 1 4 2 5 3

Post order → left, right, Root → 2 5 4 3 1



1 4 2 5 3

Pre order Traversal



1: L4 R3
4: L2 R5
2:
5:
3: R10

10 :

144. Binary Tree Preorder Traversal

Solved ✓

Easy

Topics

Companies

Given the root of a binary tree, return the preorder traversal of its nodes' values.

Example 1:

Input: root = [1,null,2,3]

Output: [1,2,3]

Preorder
↓

root, left, right

```
class Solution:
```

```
    def preorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
```

```
        def helper(root):
```

```
            if root is None:
```

```
                return
```

```
            output.append(root.val)
```

```
            helper(root.left)
```

```
            helper(root.right)
```

```
        output = []
```

```
        helper(root)
```

```
        return output
```

↓
T.C → O(n)

↓
S.C → O(n)

Call Stack space

104. Maximum Depth of Binary Tree

Solved ✓

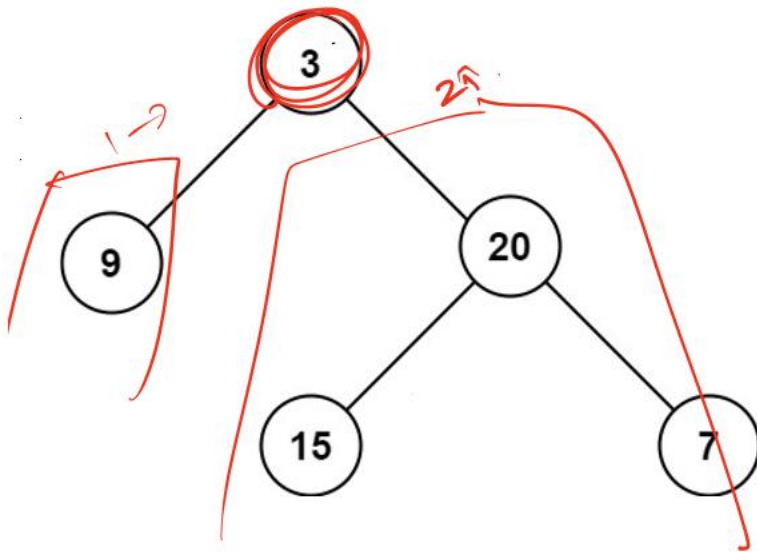
Easy

Topics

Companies

Given the root of a binary tree, return its maximum depth.

A binary tree's **maximum depth** is the number of nodes along the longest path from the root node down to the farthest leaf node.



```
class Solution:
    def maxDepth(self, root: Optional[TreeNode]) -> int:
        if root is None:
            return 0
        leftHeight = self.maxDepth(root.left)
        rightHeight = self.maxDepth(root.right)
        return 1 + max(leftHeight, rightHeight)
```

↓
T.C → $O(n)$

↓
S.C → $O(n)$ ↳ call stack space